Vau

Arquitetura do banco de dados

Diagrama de relacionamentos, todas as tabelas com campos, tipos, chaves e regras, os índices e as consultas que eles atendem, as decisões de arquitetura e como ligar o Vau a MySQL, MariaDB ou PostgreSQL.

Versão 1.0.0
Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Visão geral 4

1.1 Onde cada dado fica 4

1.2 Convenções 4

1.3 Tipos lógicos 5

1.4 Criação automática e migrações 5

02 Diagrama de relacionamentos 6

2.1 Diagrama 6

2.2 Relacionamentos 7

03 Tabelas 8

3.1 va_user 8

3.2 va_api_token 8

3.3 va_session 9

3.4 va_project 9

3.5 va_secret 11

3.6 va_webhook_delivery 11

3.7 va_build 12

3.8 va_build_step 13

3.9 va_build_log 14

3.10 va_worker 14

3.11 va_audit 15

3.12 va_rate_limit 15

3.13 va_schema_migration 15

04 Ciclos de status 16

4.1 Build (va_build.status) 16

4.2 Etapa (va_build_step.status) 17

05 Consultas e índices 18

5.1 Consultas principais 18

06 Decisões de arquitetura 19

6.1 A fila mora no banco 19

6.2 Logs em trechos 19

6.3 Segredos cifrados na aplicação 19

6.4 Sessões no banco 19

6.5 Número do build por projeto 19

6.6 Apagar sem deixar órfãos 20

Copyright (©) 2026 Erik Pimenta

07 Conectando outro banco 21
7.1 Bancos suportados 21
7.2 Preparando o banco 21
7.3 Gerando o DDL 22
7.4 Migrando do SQLite 22

08 Crescimento, retenção e backup 23
8.1 Crescimento 23
8.2 Retenção 23
8.3 Backup 23

09 DDL de referência 25
9.1 PostgreSQL 26

Copyright (©) 2026 Erik Pimenta

01

Visão geral

Este documento descreve onde o Vau guarda cada informação, como as tabelas se relacionam e o que é preciso para ligar a aplicação a um banco próprio. Serve para quem vai instalar, administrar, auditar ou integrar outros sistemas ao banco.

1.1 Onde cada dado fica

Lugar	Conteúdo	Observação
Banco relacional	Usuários, sessões, tokens, projetos, segredos cifrados, entregas de webhook, builds, etapas, logs, workers, auditoria e limite de taxa	SQLite em `storage/vau.sqlite` por padrão; MySQL, MariaDB ou PostgreSQL via `VAU_DATABASE_URL`
`storage/.chave`	Chave de 32 bytes que cifra os segredos	Criada na primeira execução quando `VAU_APP_KEY` está vazia. Sem ela, os segredos gravados não podem ser lidos
`storage/logs/vau.log`	Log técnico da aplicação em JSON Lines	Não guarda o log dos builds, que fica no banco
Volumes Docker `vau-b<ID>`	Código clonado de um build em andamento	Temporários: removidos ao fim de cada build
`storage/workspaces/`	Pastas de build do runtime local	Só existe com `VAU_RUNTIME=local` (desenvolvimento)

1.2 Convenções

Item	Regra
Prefixo	Todas as tabelas começam com `va_`, para conviver com outras aplicações no mesmo banco
Chaves	`id` numérico autoincremental; três tabelas usam chave de texto (`va_session`, `va_worker`, `va_rate_limit`)
Datas	Sempre em UTC no formato `Y-m-d H:i:s`, em qualquer banco. A conversão para o fuso de `VAU_TIMEZONE` acontece só na exibição
Booleanos	Inteiros 0 e 1 (SMALLINT, TINYINT(1))
Segredos	Texto `v1:` + base64(nonce de 24 bytes + texto cifrado) com XSalsa20-Poly1305 (libsodium secretbox)
Texto	UTF-8; no MySQL, `utf8mb4` com `utf8mb4_unicode_ci`

Copyright (©) 2026 Erik Pimenta

Item	Regra
Integridade	Chaves estrangeiras reais com `ON DELETE CASCADE` ou `SET NULL`; no SQLite, `PRAGMA foreign_keys = ON` em toda conexão

1.3 Tipos lógicos

As tabelas são descritas uma vez, com tipos lógicos, e traduzidas para cada banco. As tabelas do capítulo 3 usam estes nomes.

Tipo lógico	SQLite	MySQL / MariaDB	PostgreSQL
chave	`INTEGER PRIMARY KEY AUTOINCREMENT`	`BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY`	`BIGSERIAL PRIMARY KEY`
referência	`BIGINT`	`BIGINT UNSIGNED`	`BIGINT`
texto curto	`VARCHAR(n)`	`VARCHAR(n)`	`VARCHAR(n)`
texto	`TEXT`	`MEDIUMTEXT`	`TEXT`
texto longo	`TEXT`	`LONGTEXT`	`TEXT`
inteiro / inteiro grande	`INTEGER` / `BIGINT`	`INTEGER` / `BIGINT`	`INTEGER` / `BIGINT`
booleano 0/1	`SMALLINT`	`TINYINT(1)`	`SMALLINT`
data UTC	`TEXT` (Y-m-d H:i:s)	`DATETIME`	`TIMESTAMP(0)`

1.4 Criação automática e migrações

Na primeira conexão o Vau cria a tabela `va_schema_migration` e aplica as migrações pendentes. Cada migração é numerada, roda dentro de uma transação (exceto no MySQL, que confirma DDL sozinho) e fica registrada com a data. Rodar `php bin/vau migrate` várias vezes não altera nada. Para desligar a criação automática em produção, defina `VAU_AUTO_MIGRATE=false` e rode o comando no deploy.

Tabelas por área	
Acesso	`va_user`, `va_session`, `va_api_token`, `va_audit`, `va_rate_limit`
Projetos	`va_project`, `va_secret`
Entrada	`va_webhook_delivery`
Execução	`va_build`, `va_build_step`, `va_build_log`, `va_worker`
Controle	`va_schema_migration`

Copyright (©) 2026 Erik Pimenta

02

Diagrama de relacionamentos

As tabelas formam três blocos: o acesso ao painel em volta de va_user, a configuração em volta de va_project e a execução em volta de va_build.

2.1 Diagrama

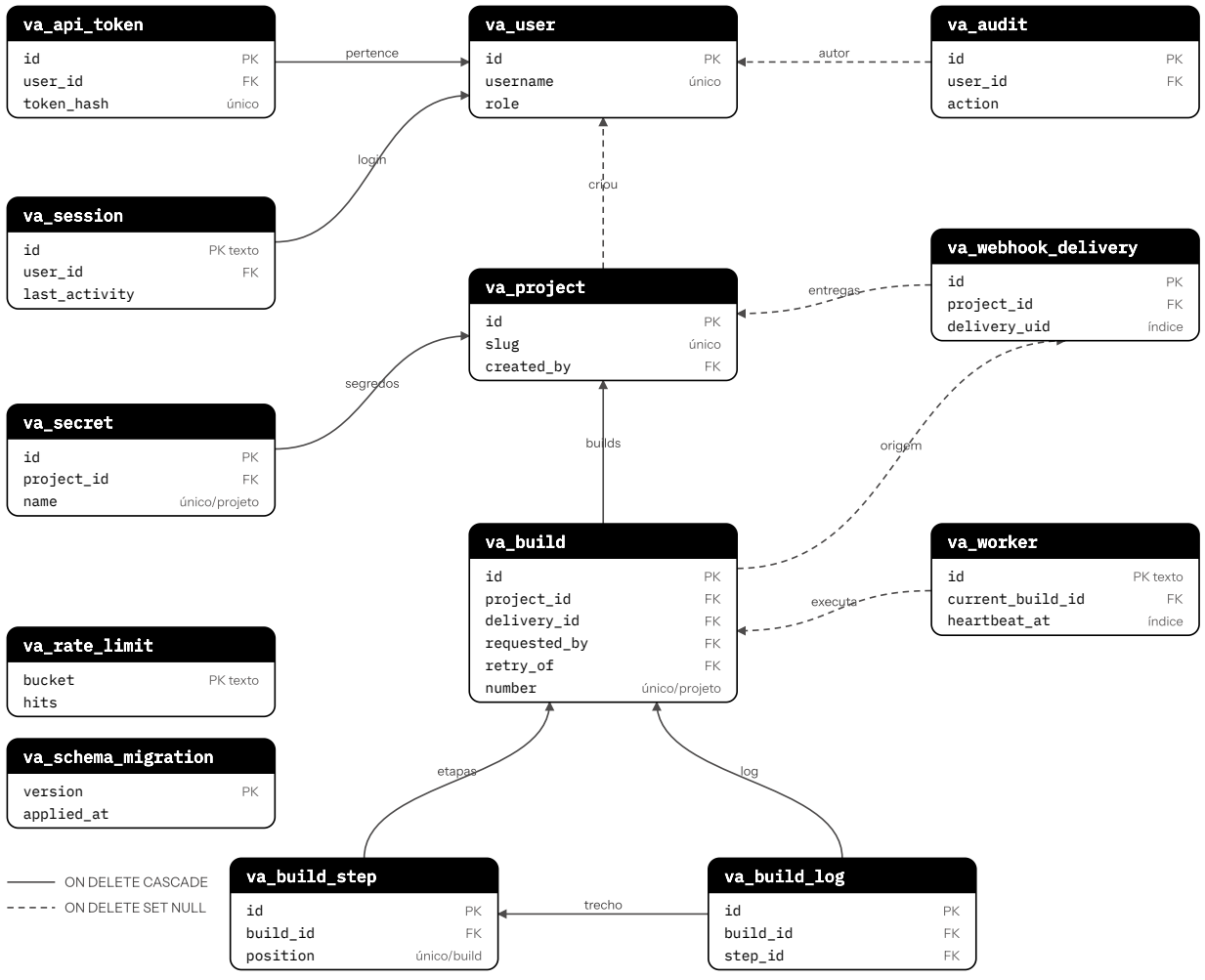


Figura 2.1 Tabelas, chaves primárias e estrangeiras. Linha contínua: apagar o pai apaga os filhos. Tracejada: o filho fica com a referência vazia. va_build.requested_by aponta para va_user e va_build.retry_of para o próprio va_build (ambas SET NULL).

Copyright (©) 2026 Erik Pimenta

2.2 Relacionamentos

Coluna	Aponta para	Cardinalidade	Ao apagar o pai	Aceita nulo
`va_api_token.user_id`	`va_user.id`	N:1	CASCADE	não
`va_session.user_id`	`va_user.id`	N:1	CASCADE	sim
`va_project.created_by`	`va_user.id`	N:1	SET NULL	sim
`va_secret.project_id`	`va_project.id`	N:1	CASCADE	não
`va_webhook_delivery.project_id`	`va_project.id`	N:1	SET NULL	sim
`va_build.project_id`	`va_project.id`	N:1	CASCADE	não
`va_build.delivery_id`	`va_webhook_delivery.id`	N:1	SET NULL	sim
`va_build.requested_by`	`va_user.id`	N:1	SET NULL	sim
`va_build.retry_of`	`va_build.id`	N:1	SET NULL	sim
`va_build_step.build_id`	`va_build.id`	N:1	CASCADE	não
`va_build_log.build_id`	`va_build.id`	N:1	CASCADE	não
`va_build_log.step_id`	`va_build_step.id`	N:1	CASCADE	sim
`va_worker.current_build_id`	`va_build.id`	N:1	SET NULL	sim
`va_audit.user_id`	`va_user.id`	N:1	SET NULL	sim

Copyright (©) 2026 Erik Pimenta

03

Tabelas

Cada tabela com suas colunas, tipos em cada banco, valores padrão e regras. O tipo lógico vem da definição única em src/Database/Schema.php, traduzida para o dialeto de cada servidor.

3.1 va_user

Contas do painel. Senhas com password_hash (Argon2id quando disponível, bcrypt como alternativa); o login é único e sempre em minúsculas.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.
`username`	`VARCHAR(60)`	não	—	Login, 3 a 60 caracteres: a-z, 0-9, ponto, hífen, sublinhado. Único.
`name`	`VARCHAR(120)`	não	—	Nome exibido no painel e na auditoria.
`password_hash`	`VARCHAR(255)`	não	—	Hash da senha; recalculado no login se o algoritmo mudar.
`role`	`VARCHAR(20)`	não	`leitor`	`admin`, `operador` ou `leitor`.
`active`	booleano 0/1	não	`1`	0 bloqueia a entrada sem apagar o histórico.
`failed_logins`	`INTEGER`	não	`0`	Tentativas erradas seguidas; zera no acerto.
`locked_until`	data UTC	sim	—	Bloqueio temporário após VAU_LOGIN_MAX_ATTEMPTS erros.
`last_login_at`	data UTC	sim	—	Último acesso com sucesso.
`created_at`	data UTC	não	—	Criação.
`updated_at`	data UTC	não	—	Última alteração.

Índices: `ux_va_user_username` (username) único.

3.2 va_api_token

Tokens da API REST. Só o SHA-256 do token é gravado; o valor completo aparece uma única vez, na criação.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Padrão	Regra
`user_id`	referência	não	—	Dono do token (herda o perfil dele). FK → `va_user` (CASCADE).
`name`	`VARCHAR(80)`	não	—	Rótulo livre, como "pipeline-externo".
`token_hash`	`VARCHAR(64)`	não	—	SHA-256 hexadecimal do token. Único.
`token_prefix`	`VARCHAR(12)`	não	—	Primeiros 12 caracteres, para reconhecer o token na lista.
`last_used_at`	data UTC	sim	—	Último uso aceito.
`expires_at`	data UTC	sim	—	Validade opcional; vazio = não expira.
`created_at`	data UTC	não	—	Criação.

Índices: `ux_va_api_token_hash` (token_hash) único; `ix_va_api_token_user` (user_id).

3.3 va_session

Sessões do painel. O cookie `vau_sessao` leva só o identificador aleatório de 256 bits; os dados ficam aqui, o que permite vários servidores atrás de um balanceador.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	`VARCHAR(128)` PK	não	—	Identificador da sessão (43 caracteres base64url).
`user_id`	referência	sim	—	Usuário logado, quando houver. FK → `va_user` (CASCADE).
`payload`	texto	não	—	Dados da sessão em JSON (usuário, token CSRF, avisos).
`ip`	`VARCHAR(45)`	sim	—	IP da última requisição.
`user_agent`	`VARCHAR(255)`	sim	—	Navegador da última requisição.
`last_activity`	`BIGINT`	não	—	Unix time da última atividade; sessões além de VAU_SESSION_MINUTES expiram.

Índices: `ix_va_session_activity` (last_activity).

3.4 va_project

Um repositório monitorado: de onde clonar, quais eventos aceitar, limites do container e avisos.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Padrão	Regra
`slug`	`VARCHAR(80)`	não	—	Identificador na URL e no webhook (`/webhook/{slug}`). Único.
`name`	`VARCHAR(120)`	não	—	Nome exibido.
`provider`	`VARCHAR(10)`	não	`github`	`github` , `gitlab` ou `generic` .
`repo_url`	`VARCHAR(500)`	não	—	URL HTTPS de clonagem.
`default_branch`	`VARCHAR(120)`	não	`main`	Branch dos builds manuais e do selo.
`webhook_secret_enc`	texto	não	—	Segredo do webhook, cifrado (v1:...).
`access_token_enc`	texto	sim	—	Token do provedor, cifrado: clone privado e status no commit.
`branch_filter`	`VARCHAR(500)`	sim	—	Padrões separados por vírgula (main, release/*, !wip/*). Vazio aceita todos.
`events`	`VARCHAR(60)`	não	`push,pull_request,tag`	Lista de `push` , `pull_request` , `tag` .
`pipeline_yaml`	texto	sim	—	Pipeline padrão, usado quando o commit não tem .vau.yml.
`timeout_minutes`	`INTEGER`	não	`30`	Tempo máximo do build (1 a 360).
`cpus`	`VARCHAR(10)`	não	`1.0`	Limite de CPU por container (--cpus).
`memory`	`VARCHAR(10)`	não	`1g`	Limite de memória (--memory e --memory-swap).
`network`	`VARCHAR(20)`	não	`bridge`	`bridge` (com internet) ou `none` .
`auto_cancel`	booleano 0/1	não	`1`	Cancela builds da mesma branch ainda na fila quando chega commit novo.
`allow_deploy`	booleano 0/1	não	`1`	Permite a etapa deploy do pipeline.
`report_status`	booleano 0/1	não	`1`	Publica o status no commit (precisa do token).
`notify_url`	`VARCHAR(500)`	sim	—	Webhook de chat (HTTPS).
`notify_on`	`VARCHAR(10)`	não	`falhas`	`falhas` , `todas` ou `nunca` .
`active`	booleano 0/1	não	`1`	0 recusa webhooks (404) sem apagar o histórico.
`created_by`	referência	sim	—	Quem cadastrou. FK → `va_user` (SET NULL).
`created_at`	data UTC	não	—	Criação.
`updated_at`	data UTC	não	—	Última alteração.

Índices: `ux_va_project_slug` (slug) único.

Copyright (©) 2026 Erik Pimenta

3.5 va_secret

Variáveis secretas do projeto, injetadas como ambiente dos containers e mascaradas no log.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.
`project_id`	referência	não	—	Projeto dono. FK → `va_project` (CASCADE).
`name`	`VARCHAR(80)`	não	—	Nome da variável (A-Z, 0-9, _). Único por projeto. Prefixo VAU_ é reservado.
`value_enc`	texto	não	—	Valor cifrado (v1:...). Nunca volta para o painel.
`deploy_only`	booleano 0/1	não	`0`	1 = só chega na etapa de deploy.
`created_at`	data UTC	não	—	Criação.
`updated_at`	data UTC	não	—	Última troca de valor.

Índices: `ux_va_secret_name` (project_id, name) único.

3.6 va_webhook_delivery

Toda chamada em /webhook, aceita ou recusada, com cabeçalhos e corpo para conferência e reenvio.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.
`project_id`	referência	sim	—	Projeto do slug, se existir. FK → `va_project` (SET NULL).
`provider`	`VARCHAR(10)`	não	—	`github`, `gitlab` ou `unknown`.
`event`	`VARCHAR(60)`	não	—	X-GitHub-Event ou X-Gitlab-Event.
`delivery_uid`	`VARCHAR(100)`	não	—	X-GitHub-Delivery, X-Gitlab-Event-UUID ou sha1 do corpo; usado contra entregas repetidas.
`signature_ok`	booleano 0/1	não	`0`	1 quando a assinatura conferiu.
`status_code`	`INTEGER`	não	—	Código HTTP devolvido (202 = build criado).
`outcome`	`VARCHAR(255)`	não	—	Explicação do resultado.
`headers`	texto	não	—	Cabeçalhos em JSON (X-Gitlab-Token, Authorization e Cookie mascarados).
`payload`	texto longo	não	—	Corpo recebido (até 256 KB guardados).
`payload_size`	`INTEGER`	não	`0`	Tamanho original do corpo, em bytes.
`ip`	`VARCHAR(45)`	sim	—	IP de origem.
`received_at`	data UTC	não	—	Recebimento.

Copyright (©) 2026 Erik Pimenta

Índices: `ix_va_delivery_project` (project_id, received_at); `ix_va_delivery_uid` (delivery_uid); `ix_va_delivery_received` (received_at).

3.7 va_build

Uma execução do pipeline para um commit. Também é a fila: status `queued` espera um worker.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador global.
`project_id`	referência	não	—	Projeto. FK → `va_project` (CASCADE).
`number`	`INTEGER`	não	—	Número sequencial dentro do projeto (#1, #2…). Único por projeto.
`status`	`VARCHAR(12)`	não	`queued`	`queued`, `running`, `success`, `failed`, `error`, `canceled`.
`trigger_kind`	`VARCHAR(16)`	não	—	`push`, `pull_request`, `tag`, `manual`, `retry`, `api`.
`event`	`VARCHAR(40)`	não	—	Evento de origem.
`git_ref`	`VARCHAR(255)`	não	—	Referência buscada no clone (refs/heads/x, refs/pull/N/head, refs/merge-requests/N/head, refs/tags/v1).
`branch`	`VARCHAR(255)`	sim	—	Branch (ou branch de origem do PR).
`tag`	`VARCHAR(255)`	sim	—	Tag, em builds de tag.
`commit_sha`	`VARCHAR(64)`	sim	—	Commit exato a construir.
`commit_message`	`VARCHAR(500)`	sim	—	Mensagem do commit ou título do PR (500).
`commit_author`	`VARCHAR(120)`	sim	—	Autor.
`commit_url`	`VARCHAR(500)`	sim	—	Link do commit ou do PR.
`pr_number`	`INTEGER`	sim	—	Número do PR/MR.
`clone_url`	`VARCHAR(500)`	sim	—	URL informada pelo provedor (só referência; o clone usa repo_url do projeto).
`is_fork`	booleano 0/1	não	`0`	1 = PR de fork: segredos não são entregues.
`is_deploy`	booleano 0/1	não	`0`	1 quando a etapa de deploy rodou.
`delivery_id`	referência	sim	—	Entrega que gerou o build. FK → `va_webhook_delivery` (SET NULL).
`requested_by`	referência	sim	—	Usuário de builds manuais. FK → `va_user` (SET NULL).
`retry_of`	referência	sim	—	Build original de uma repetição. FK → `va_build` (SET NULL).
`worker_id`	`VARCHAR(64)`	sim	—	Worker que reservou o build.

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Padrão	Regra
`pipeline_file`	`VARCHAR(60)`	sim	—	Arquivo usado (.vau.yml ou "padrão do projeto").
`pipeline_json`	texto	sim	—	Pipeline validado, congelado em JSON.
`queued_at`	data UTC	não	—	Entrada na fila.
`started_at`	data UTC	sim	—	Reserva pelo worker.
`finished_at`	data UTC	sim	—	Fim.
`duration_ms`	`BIGINT`	sim	—	Duração total.
`exit_code`	`INTEGER`	sim	—	Código de saída da etapa decisiva.
`error_message`	`VARCHAR(500)`	sim	—	Motivo de falha, erro ou cancelamento.
`cancel_requested`	booleano 0/1	não	`0`	1 = cancelar; o worker confere a cada segundo.
`log_bytes`	`BIGINT`	não	`0`	Tamanho do log gravado.

Índices: `ux_va_build_number` (project_id, number) único; `ix_va_build_status` (status, id); `ix_va_build_branch` (project_id, branch, status); `ix_va_build_queued` (queued_at).

3.8 va_build_step

Etapas de um build: o clone, cada item de steps e o deploy. Criadas antes de rodar, para o painel mostrar o plano completo.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.
`build_id`	referência	não	—	Build. FK → `va_build` (CASCADE).
`position`	`INTEGER`	não	—	Ordem (0 = clone). Única por build.
`name`	`VARCHAR(120)`	não	—	Nome da etapa.
`kind`	`VARCHAR(10)`	não	`step`	`clone`, `step` ou `deploy`.
`image`	`VARCHAR(255)`	sim	—	Imagem Docker usada.
`command`	texto	sim	—	Script executado com sh -e.
`status`	`VARCHAR(10)`	não	`pending`	`pending`, `running`, `success`, `failed`, `skipped`, `canceled`.
`exit_code`	`INTEGER`	sim	—	Código de saída (124 = tempo limite, 130 = cancelado).
`started_at`	data UTC	sim	—	Início.
`finished_at`	data UTC	sim	—	Fim.
`duration_ms`	`BIGINT`	sim	—	Duração.

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Padrão	Regra
`note`	`VARCHAR(255)`	sim	—	Motivo de pulo ou interrupção.

Índices: `ux_va_step_position` (build_id, position) único.

3.9 va_build_log

Saída dos builds em trechos de até 16 KB, já sem códigos ANSI e com segredos mascarados.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador; também é o cursor da leitura incremental do painel.
`build_id`	referência	não	—	Build. FK → `va_build` (CASCADE).
`step_id`	referência	sim	—	Etapa (vazio = mensagens gerais do Vau). FK → `va_build_step` (CASCADE).
`stream`	`VARCHAR(3)`	não	`out`	`out`, `err` ou `sys` (linhas do próprio Vau).
`content`	texto longo	não	—	Texto do trecho.
`created_at`	data UTC	não	—	Gravação.

Índices: `ix_va_log_build` (build_id, id).

3.10 va_worker

Workers conectados à fila, com batimento a cada laço (e a cada 15 s durante um build, com pcntl).

Coluna	Tipo	Nulo	Padrão	Regra
`id`	`VARCHAR(64) PK`	não	—	host-pid-aleatório.
`hostname`	`VARCHAR(120)`	não	—	Máquina.
`pid`	`INTEGER`	não	—	Processo.
`status`	`VARCHAR(10)`	não	`idle`	`idle`, `busy` ou `stopped`.
`current_build_id`	referência	sim	—	Build em execução. FK → `va_build` (SET NULL).
`builds_done`	`INTEGER`	não	`0`	Builds concluídos desde o início.
`started_at`	data UTC	não	—	Início do processo.
`heartbeat_at`	data UTC	não	—	Último sinal de vida.

Índices: `ix_va_worker_heartbeat` (heartbeat_at).

Copyright (©) 2026 Erik Pimenta

3.11 va_audit

Trilha de auditoria das ações no painel e na API.

Coluna	Tipo	Nulo	Padrão	Regra
`id`	chave	não	—	Identificador.
`user_id`	referência	sim	—	Quem fez (vazio = sistema ou anônimo). FK → `va_user` (SET NULL).
`action`	`VARCHAR(60)`	não	—	Ação, como `build.cancel` ou `secret.put`.
`target_type`	`VARCHAR(30)`	sim	—	Tipo do alvo (project, build, user...).
`target_id`	`VARCHAR(64)`	sim	—	Identificador do alvo.
`details`	texto	sim	—	Detalhes em JSON (nunca valores de segredos).
`ip`	`VARCHAR(45)`	sim	—	IP de origem.
`created_at`	data UTC	não	—	Momento.

Índices: `ix_va_audit_created` (created_at).

3.12 va_rate_limit

Contadores do limite de taxa por janela fixa (login e webhooks).

Coluna	Tipo	Nulo	Padrão	Regra
`bucket`	`VARCHAR(120)` `PK`	não	—	SHA-256 truncado de "regra:ip".
`hits`	`INTEGER`	não	`0`	Requisições na janela atual.
`window_start`	`BIGINT`	não	—	Unix time do início da janela.

3.13 va_schema_migration

Controle das migrações aplicadas. Criada antes de todas as outras.

Coluna	Tipo	Nulo	Padrão	Regra
`version`	`INTEGER PK`	não	—	Número da migração.
`applied_at`	data UTC	não	—	Quando foi aplicada.

Copyright (©) 2026 Erik Pimenta

04

Ciclos de status

O status do build e das etapas muda sempre na mesma direção. Um build terminado nunca volta para a fila: repetir cria um build novo com retry_of apontando para o anterior.

4.1 Build (va_build.status)

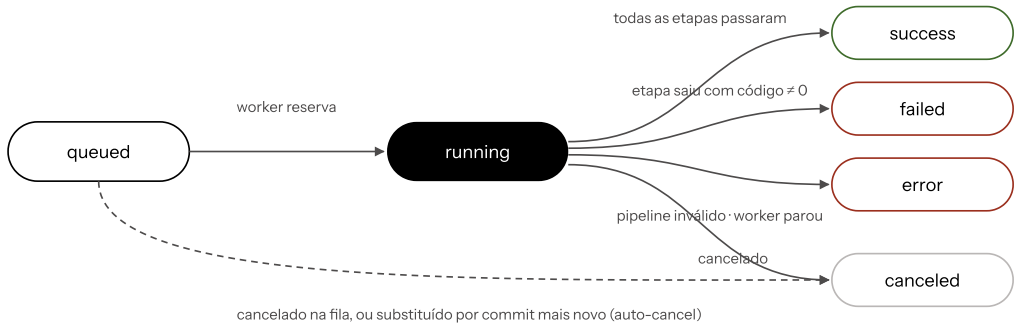


Figura 4.1 Ciclo de vida do build. A reserva pelo worker é a única passagem para running.

Status	Quando	Quem grava
`queued`	Webhook aceito, build manual, API ou repetição	Painel, API, CLI
`running`	Um worker reservou o build	Worker (`UPDATE ... WHERE status = 'queued'`)
`success`	Todas as etapas (e o deploy, se liberado) saíram com código 0	Worker
`failed`	Uma etapa saiu com código diferente de 0, o clone falhou ou o tempo limite acabou	Worker
`error`	Pipeline ausente ou inválido, falha interna, ou worker que parou de dar sinal de vida	Worker
`canceled`	Cancelado no painel ou na API, ou substituído por commit mais novo da mesma branch	Painel, API, fila

Copyright (©) 2026 Erik Pimenta

4.2 Etapa (va_build_step.status)

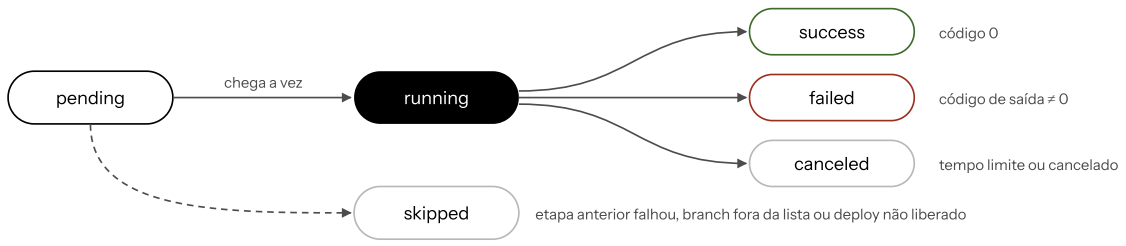


Figura 4.2 Ciclo de vida das etapas. skipped explica o motivo em note.

Copyright (©) 2026 Erik Pimenta

05

Consultas e índices

Cada índice existe por causa de uma consulta frequente da aplicação.

5.1 Consultas principais

Uso	Consulta	Índice
Worker procura trabalho	`WHERE status = 'queued' ORDER BY id LIMIT 10`	`ix_va_build_status`
Worker reserva o build	`UPDATE va_build SET status='running' WHERE id = ? AND status = 'queued'`	chave primária
Auto-cancel da branch	`WHERE project_id = ? AND branch = ? AND status = 'queued'`	`ix_va_build_branch`
Próximo número do build	`MAX(number) WHERE project_id = ?`	`ux_va_build_number`
Log ao vivo no painel	`WHERE build_id = ? AND id > ? ORDER BY id`	`ix_va_log_build`
Etapas do build	`WHERE build_id = ? ORDER BY position`	`ux_va_step_position`
Entrega repetida	`WHERE project_id = ? AND delivery_uid = ? AND status_code = 202`	`ix_va_delivery_uid`
Webhook encontra o projeto	`WHERE slug = ?`	`ux_va_project_slug`
Login	`WHERE username = ?`	`ux_va_user_username`
Token da API	`WHERE token_hash = sha256(?)`	`ux_va_api_token_hash`
Painel e /metrics	`WHERE queued_at >= ?` agrupado por status	`ix_va_build_queued`
Builds órfãos	build `running` com worker sem batimento recente	`ix_va_worker_heartbeat`
Limpeza de sessões	`DELETE WHERE last_activity < ?`	`ix_va_session_activity`
Auditoria e retenção	`ORDER BY id DESC`, `WHERE received_at < ?`	`ix_va_audit_created`, `ix_va_delivery_received`

Copyright (©) 2026 Erik Pimenta

06

Decisões de arquitetura

Por que o banco é como é.

6.1 A fila mora no banco

Não há Redis nem RabbitMQ. A fila é a tabela `va_build` e a reserva é uma atualização condicional: só uma conexão consegue mudar a linha de `queued` para `running`, e o worker confere `rowCount() = 1`. Isso funciona igual em SQLite, MySQL e PostgreSQL, sem travas longas, e permite vários workers em várias máquinas apontando para o mesmo banco. Um build reservado por um worker que morreu é encerrado com `error` pela rotina de recuperação (batimento mais antigo que `VAU_WORKER_STALE_SECONDS`).

6.2 Logs em trechos

O worker junta a saída por até meio segundo ou 16 KB e grava um trecho por vez em `va_build_log`, em vez de uma linha por registro. O painel lê só os trechos com `id` maior que o último recebido. O total por build é limitado por `VAU_MAX_LOG_MB`; quando passa, o log é cortado com aviso e o build continua.

6.3 Segredos cifrados na aplicação

Segredos de webhook, tokens de acesso e variáveis secretas são cifrados antes de chegar ao banco. Um backup ou uma réplica do banco, sozinho, não expõe nenhum valor. A chave fica fora do banco (`VAU_APP_KEY` ou `storage/.chave`).

6.4 Sessões no banco

Guardar a sessão em `va_session` permite rodar vários servidores do painel atrás de um balanceador sem afinidade, e encerrar o acesso de alguém apagando uma linha.

6.5 Número do build por projeto

O número (#1, #2…) é calculado como `MAX(number) + 1` dentro de uma transação e protegido pelo índice único `(project_id, number)`. Se dois webhooks simultâneos disputarem o mesmo número, um deles recebe violação de unicidade e tenta de novo, até cinco vezes, com espera aleatória.

Copyright (©) 2026 Erik Pimenta

6.6 Apagar sem deixar órfãos

Apagar um projeto apaga segredos, builds, etapas e logs (CASCADE). Entregas de webhook e auditoria ficam, com a referência vazia (SET NULL), para não perder o rastro do que aconteceu.

Copyright (©) 2026 Erik Pimenta

07

Conectando outro banco

O SQLite atende um servidor com poucos workers. Para vários servidores do painel, muitos workers ou alta disponibilidade, use MySQL, MariaDB ou PostgreSQL.

7.1 Bancos suportados

Banco	Versão mínima	Extensão PHP	`VAU_DATABASE_URL`
SQLite	3.35	pdo_sqlite	`sqlite:storage/vau.sqlite`
MySQL	8.0	pdo_mysql	`mysql://vau:senha@host:3306/vau`
MariaDB	10.6	pdo_mysql	`mariadb://vau:senha@host:3306/vau`
PostgreSQL	13	pdo_pgsql	`pgsql://vau:senha@host:5432/vau`

Caracteres especiais da senha vão codificados na URL (por exemplo, `@` vira `%40`). Em toda conexão o Vau fixa UTC (`SET time_zone = '+00:00'` no MySQL, `SET TIME ZONE 'UTC'` no PostgreSQL) e, no MySQL, o modo estrito.

7.2 Preparando o banco

```
-- PostgreSQL
CREATE USER vau WITH PASSWORD 'troque-esta-senha';
CREATE DATABASE vau OWNER vau ENCODING 'UTF8';

-- MySQL / MariaDB
CREATE DATABASE vau CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'vau'@'%' IDENTIFIED BY 'troque-esta-senha';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, REFERENCES ON vau.* TO 'vau'@'%';
```

Depois das migrações, o usuário do dia a dia só precisa de SELECT, INSERT, UPDATE e DELETE. CREATE, INDEX e REFERENCES servem para `php bin/vau migrate`; em ambientes mais restritos, rode as migrações com outro usuário e desligue `VAU_AUTO_MIGRATE`.

Copyright (©) 2026 Erik Pimenta

7.3 Gerando o DDL

```
php bin/vau schema:dump pgsql  > schema-postgresql.sql
php bin/vau schema:dump mysql  > schema-mysql.sql
php bin/vau schema:dump sqlite > schema-sqlite.sql
```

Os três arquivos já gerados estão em `docs/`. Quem prefere criar a estrutura pelo DBA aplica o DDL e registra a versão: `INSERT INTO va_schema_migration (version, applied_at) VALUES (1, CURRENT_TIMESTAMP)`.

7.4 Migrando do SQLite

1. Pare o painel e os workers.
2. Aponte `VAU_DATABASE_URL` para o banco novo e rode `php bin/vau migrate` para criar as tabelas.
3. Copie os dados tabela por tabela, na ordem das chaves estrangeiras: `va_user`, `va_api_token`, `va_project`, `va_secret`, `va_webhook_delivery`, `va_build`, `va_build_step`, `va_build_log`, `va_audit` (sessões, workers e limites de taxa podem ficar para trás).
4. No PostgreSQL, ajuste as sequências: `SELECT setval('va_build_id_seq', (SELECT MAX(id) FROM va_build))`, e o mesmo para as outras tabelas com `id` numérico.
5. Mantenha a mesma chave (`storage/.chave` ou `VAU_APP_KEY`): sem ela, os segredos copiados não abrem.

Copyright (©) 2026 Erik Pimenta

08

Crescimento, retenção e backup

O que mais cresce é o log dos builds. Uma rotina de limpeza e um backup diário bastam para a maioria das equipes.

8.1 Crescimento

Tabela	Cresce com	Ordem de grandeza
`va_build_log`	Saída dos builds	Até `VAU_MAX_LOG_MB` por build (10 MB por padrão); builds comuns ficam entre 5 e 200 KB
`va_webhook_delivery`	Cada push, PR e ping	Até 256 KB de corpo por entrega
`va_build` e `va_build_step`	Builds	Poucos KB por build, incluindo o pipeline congelado em JSON
Demais	Uso do painel	Desprezível

8.2 Retenção

```
# apaga builds terminados e entregas com mais de 30 dias (VAU_LOG_RETENTION_DAYS)
php bin/vau prune
php bin/vau prune --dias=90

# agendado todo dia às 03:10 (cron do Linux)
10 3 * * * cd /opt/vau && php bin/vau prune >> storage/logs/prune.log 2>&1
```

O `prune` só apaga builds em estado final. Etapas e logs saem junto (CASCADE). Entregas ligadas a um build que ficou são preservadas.

8.3 Backup

Banco	Comando
SQLite	`sqlite3 storage/vau.sqlite ".backup storage/backup-$(date +%F).sqlite"` (seguro com o Vau rodando, graças ao WAL)
PostgreSQL	`pg_dump -Fc -U vau vau > vau-$(date +%F).dump`
MySQL / MariaDB	`mysqldump --single-transaction vau > vau-$(date +%F).sql`

Copyright (©) 2026 Erik Pimenta

> Guarde a chave (`storage/.chave` ou o valor de `VAU_APP_KEY`) num cofre separado do backup do banco. Um backup sem a chave não abre os segredos; a chave sem o backup não serve para nada.

Copyright (©) 2026 Erik Pimenta

09

DDL de referência

Trecho do DDL do PostgreSQL gerado por php bin/vau schema:dump pgsql. Os arquivos completos dos três bancos estão em docs/.

Copyright (©) 2026 Erik Pimenta

9.1 PostgreSQL

```
CREATE TABLE IF NOT EXISTS va_project (
  id BIGSERIAL PRIMARY KEY,
  slug VARCHAR(80) NOT NULL,
  name VARCHAR(120) NOT NULL,
  provider VARCHAR(10) NOT NULL DEFAULT 'github',
  repo_url VARCHAR(500) NOT NULL,
  default_branch VARCHAR(120) NOT NULL DEFAULT 'main',
  webhook_secret_enc TEXT NOT NULL,
  access_token_enc TEXT NULL,
  branch_filter VARCHAR(500) NULL,
  events VARCHAR(60) NOT NULL DEFAULT 'push,pull_request,tag',
  pipeline_yaml TEXT NULL,
  timeout_minutes INTEGER NOT NULL DEFAULT 30,
  cpus VARCHAR(10) NOT NULL DEFAULT 1.0,
  memory VARCHAR(10) NOT NULL DEFAULT '1g',
  network VARCHAR(20) NOT NULL DEFAULT 'bridge',
  auto_cancel SMALLINT NOT NULL DEFAULT 1,
  allow_deploy SMALLINT NOT NULL DEFAULT 1,
  report_status SMALLINT NOT NULL DEFAULT 1,
  notify_url VARCHAR(500) NULL,
  notify_on VARCHAR(10) NOT NULL DEFAULT 'falhas',
  active SMALLINT NOT NULL DEFAULT 1,
  created_by BIGINT NULL,
  created_at TIMESTAMP(0) NOT NULL,
  updated_at TIMESTAMP(0) NOT NULL,
  FOREIGN KEY (created_by) REFERENCES va_user(id) ON DELETE SET NULL
);

CREATE UNIQUE INDEX IF NOT EXISTS ux_va_project_slug ON va_project (slug);

CREATE TABLE IF NOT EXISTS va_build (
  id BIGSERIAL PRIMARY KEY,
  project_id BIGINT NOT NULL,
  number INTEGER NOT NULL,
  status VARCHAR(12) NOT NULL DEFAULT 'queued',
  trigger_kind VARCHAR(16) NOT NULL,
  event VARCHAR(40) NOT NULL,
  git_ref VARCHAR(255) NOT NULL,
  branch VARCHAR(255) NULL,
  tag VARCHAR(255) NULL,
  commit_sha VARCHAR(64) NULL,
  commit_message VARCHAR(500) NULL,
  commit_author VARCHAR(120) NULL,
  commit_url VARCHAR(500) NULL,
  pr_number INTEGER NULL,
  clone_url VARCHAR(500) NULL,
  is_fork SMALLINT NOT NULL DEFAULT 0,
  is_deploy SMALLINT NOT NULL DEFAULT 0,
  delivery_id BIGINT NULL,
  requested_by BIGINT NULL,
  retry_of BIGINT NULL,
  worker_id VARCHAR(64) NULL,
  pipeline_file VARCHAR(60) NULL,
  pipeline_json TEXT NULL,
  queued_at TIMESTAMP(0) NOT NULL,
  started_at TIMESTAMP(0) NULL,
```

Copyright (©) 2026 Erik Pimenta

```
  finished_at TIMESTAMP(0) NULL,
  duration_ms BIGINT NULL,
  exit_code INTEGER NULL,
  error_message VARCHAR(500) NULL,
  cancel_requested SMALLINT NOT NULL DEFAULT 0,
  log_bytes BIGINT NOT NULL DEFAULT 0,
  FOREIGN KEY (project_id) REFERENCES va_project(id) ON DELETE CASCADE,
  FOREIGN KEY (delivery_id) REFERENCES va_webhook_delivery(id) ON DELETE SET NULL,
  FOREIGN KEY (requested_by) REFERENCES va_user(id) ON DELETE SET NULL,
  FOREIGN KEY (retry_of) REFERENCES va_build(id) ON DELETE SET NULL
);

CREATE UNIQUE INDEX IF NOT EXISTS ux_va_build_number ON va_build (project_id, number);

CREATE INDEX IF NOT EXISTS ix_va_build_status ON va_build (status, id);

CREATE INDEX IF NOT EXISTS ix_va_build_branch ON va_build (project_id, branch, status);

CREATE INDEX IF NOT EXISTS ix_va_build_queued ON va_build (queued_at);
```

Copyright (©) 2026 Erik Pimenta